



AJAX

Presented by
Nguyễn Đức Thái
Ho Chi Minh City University of Technology

Introduction

- AJAX is a developer's **dream**, because we can:
 - **Read data** from a web server - after the page has loaded.
 - **Update a web page** without reloading the page
 - **Send data** to a web server - in the background

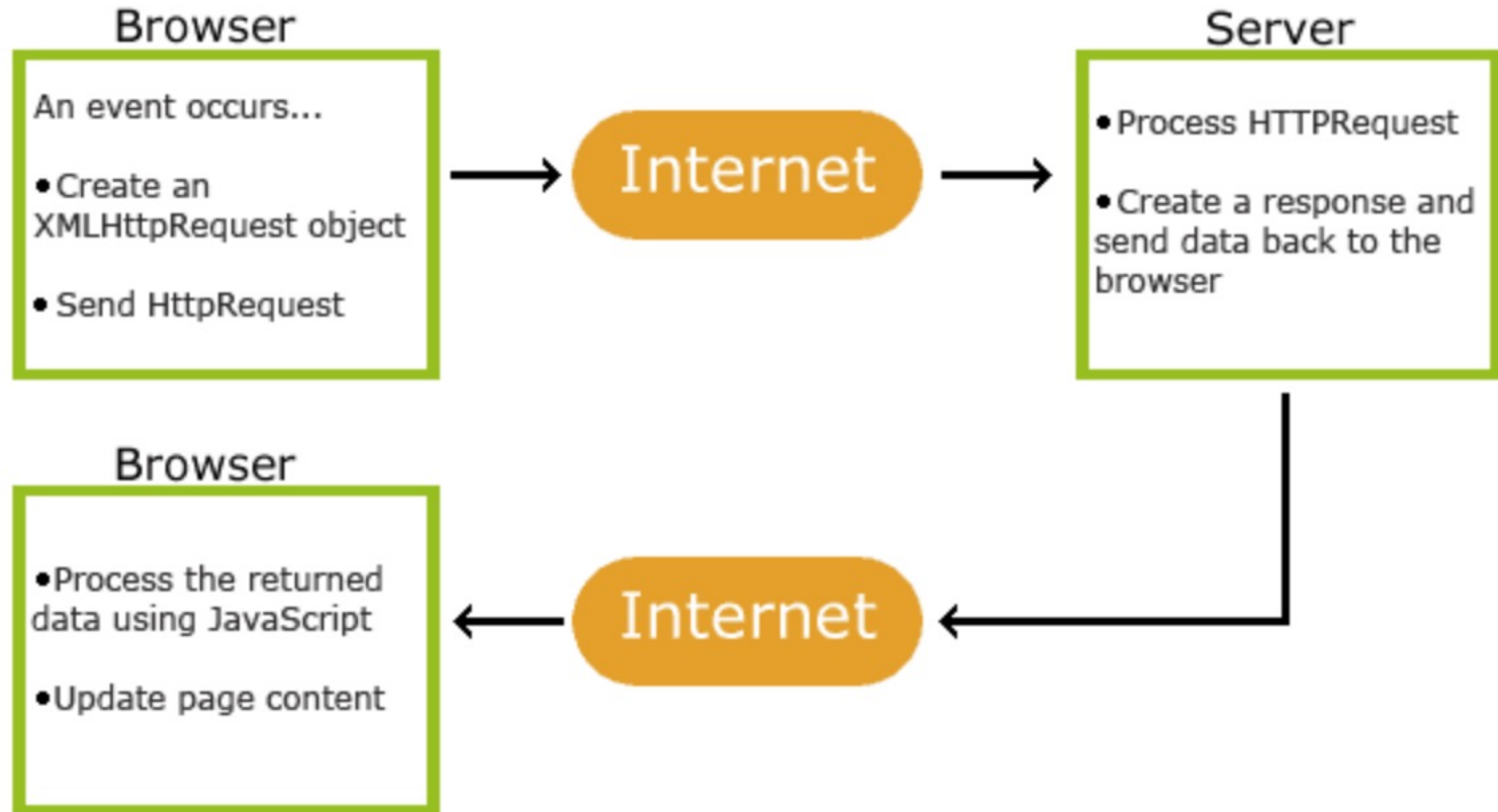
What is AJAX?

- AJAX = **A**ynchronous **J**avaScript **A**nd **X**ML.
- AJAX is **not** a programming language.
- AJAX just uses a *combination of*:
 - A **browser** built-in XMLHttpRequest object (to request data from a web server)
 - **JavaScript** and **HTML DOM** (to display or use the data)

What is AJAX?

- AJAX is a *misleading* name.
- AJAX applications might **use XML to transport data**, but it is equally common **to transport data as plain text** or **JSON text**.

How AJAX Works?



Create an XMLHttpRequest Object

- All *modern* browsers (Chrome, Firefox, IE, Edge, Safari, Opera) have a **built-in** XMLHttpRequest object.
- Syntax for creating an XMLHttpRequest object:

```
variable = new XMLHttpRequest();
```

Send A Request

The **XMLHttpRequest** object can be used to exchange data with a web server behind the scenes. All modern browsers (*Chrome, Firefox, IE, Edge, Safari, Opera*) have a built-in **XMLHttpRequest** object.

To send a request to a server, you can use the **open()** and **send()** methods of the **XMLHttpRequest** object

```
xhttp.open("GET", "ajax_info.txt");  
xhttp.send();
```

```
// Create an XMLHttpRequest object  
const xhttp = new XMLHttpRequest();  
  
// Define a callback function  
xhttp.onload = function() {  
    // Here you can use the Data  
}  
  
// Send a request  
xhttp.open("GET", "ajax_info.txt");  
xhttp.send();
```

Callback Function

the **callback function** should contain the code to execute *when the response is ready*.

```
// Create an XMLHttpRequest object
var xhttp = new XMLHttpRequest();

// Define a callback function
xhttp.onload = function() {
    // Here you can use the Data
}

// Send a request
xhttp.open("GET", "ajax_info.txt");
xhttp.send();
```


Send A Request

the **callback function** should contain the code to execute *when the response is ready*.

```
// Create an XMLHttpRequest object
var xhttp = new XMLHttpRequest();

// Define a callback function
xhttp.onload = function() {
    // Here you can use the Data
}

// Send a request
xhttp.open("GET", "ajax_info.txt");
xhttp.send();
```

XMLHttpRequest

```
xhttp.open("GET", "ajax_info.txt");  
xhttp.send();
```

Method

Description

`open(method, url, async)`

Specifies the type of request

method: the type of request: GET or POST

url: the server (file) location

async: true (asynchronous) or false (synchronous)

`send()`

Sends the request to the server (used for GET)

`send(string)`

Sends the request to the server (used for POST)

- address to **a file** on a server
- can be **any kind of file**, like **.txt** and **.xml**, or server scripting files like **.asp** and **.php**

XMLHttpRequest Object

Method	Description
<code>new XMLHttpRequest()</code>	Creates a new XMLHttpRequest object
<code>abort()</code>	Cancels the current request
<code>getAllResponseHeaders()</code>	Returns header information
<code>getResponseHeader()</code>	Returns specific header information
<code>open(method, url, async, user, psw)</code> <pre>xhttp.open("GET", "ajax_info.txt"); xhttp.send();</pre>	Specifies the request <i>method</i> : the request type GET or POST <i>url</i> : the file location <i>async</i> : true (asynchronous) or false (synchronous) <i>user</i> : optional user name <i>psw</i> : optional password
<code>send()</code>	Sends the request to the server Used for GET requests
<code>send(string)</code>	Sends the request to the server. Used for POST requests
<code>setRequestHeader()</code>	Adds a label/value pair to the header to be sent

XMLHttpRequest Object Properties

Property	Description
onload	Defines a function to be called when the request is recieved (loaded)
onreadystatechange	Defines a function to be called when the readyState property changes
readyState	Holds the status of the XMLHttpRequest. 0: request not initialized 1: server connection established 2: request received 3: processing request 4: request finished and response is ready
responseText	Returns the response data as a string
responseXML	Returns the response data as XML data
status	Returns the status-number of a request 200: "OK" 403: "Forbidden" 404: "Not Found" For a complete list go to the Http Messages Reference
statusText	Returns the status-text (e.g. "OK" or "Not Found")

```
const xhttp = new XMLHttpRequest();  
xhttp.onreadystatechange = function() {  
  if (this.readyState == 4 && this.status == 200) {  
    document.getElementById("demo").innerHTML =  
      this.responseText;  
  }  
}
```

Asynchronous – True or False?

- Server requests should be sent **asynchronously**.
- The **async** parameter of the `open()` method should be set to `true`:

```
xhttp.open("GET", "ajax_test.asp", true);
```

- By sending asynchronously, the JavaScript *does not have to wait* for the server response, but can instead:
 - execute other scripts while waiting for server response
 - deal with the response after the response is ready

AJAX Response

- Server response properties:

Property	Description
responseText	get the response data as a string
responseXML	get the response data as XML data

- The **responseText** property returns the server response as a JavaScript **string**, and you can use it accordingly:

```
document.getElementById("demo").innerHTML = xhttp.responseText;
```

- The **responseXML** property returns the server response as an **XML DOM object**.

References

- W3 Schools – [w3schools.com](https://www.w3schools.com)